

Backend presentation

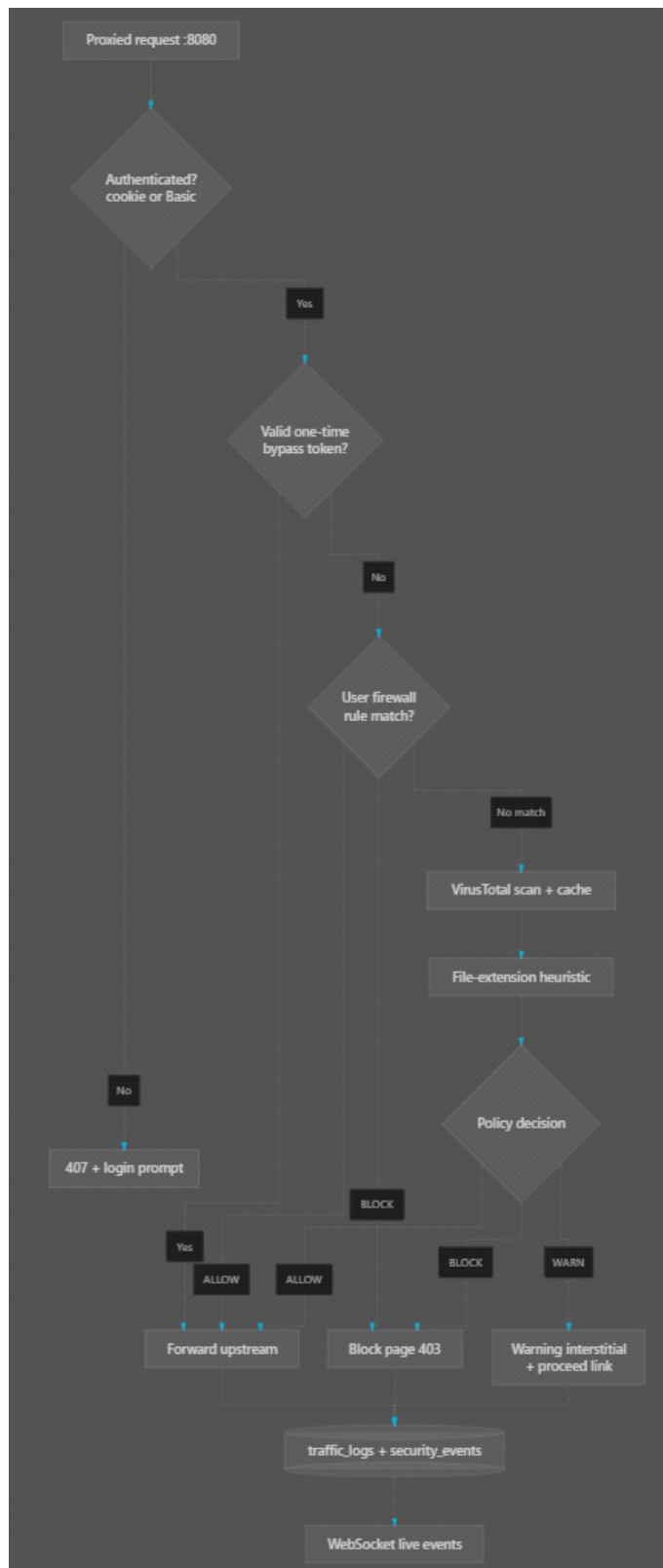
How GuardMe works

GuardMe is a personal secure web gateway: a NestJS modular monolith that runs two listeners in one process:

- 3000 - NestJS API (+ WebSocket) - Dashboard backend: auth, vault, rules, SIEM queries, health
- 8080 - Forward proxy - Inspects and controls all browser traffic



The request lifecycle:



For HTTPS, the browser sends CONNECT host:443; GuardMe scans the hostname reputation, then opens a TCP tunnel if allowed (it can't see the path without TLS interception).

Security review:

- **Session JWT** (httpOnly cookie) + **server-side session record** + **device fingerprint** (IP + User-Agent) + **re-authentication window** for sensitive actions.
- **Rate limiting** on both the API and the proxy; Helmet security headers on the API.

Proxy **brute-force protection**: an in-memory per-IP fixed-window limiter (same limits as the dashboard auth endpoints) that blocks repeated failed `Proxy-Authorization: Basic` attempts before any password hashing or DB work runs.

- **Per-user data isolation** enforced on every query (traffic logs, security events, firewall rules, vault credentials all scoped by user id).
- **Constant-time authentication**: authentication runs a full Argon2id verification even when the username does not exist (against a dummy hash), so login and proxy auth take the same time whether or not a username is valid.
- **Threat intelligence integration** - VirusTotal URL reputation (cached lookup → submit → poll), mapped to a verdict + risk score. A fail-open policy: scan errors/timeouts allow as `UNVERIFIED` and are logged, so the gateway never bricks browsing; internal proxy errors fail closed with a block page.
- **Policy engine** — ALLOW / WARN / BLOCK from verdict + file heuristic + user rules.
- **SIEM** — every request → traffic_logs; every anomaly → security_events; both streamed live over WebSocket.
- **Encryption vault**: Argon2id KDF (per-user salt) → 32-byte per-user key → AES-256-GCM per credential with a fresh random IV; derived key kept in memory only and zero-filled on lock/logout.
Login password → Argon2id verify → if valid, the same password is run through Argon2id-as-KDF with a per-user salt to derive a 32-byte key, cached in memory only. Each credential's password is AES-256-GCM encrypted with a fresh random IV. Reading a credential additionally requires the vault to be unlocked and a recent re-auth.
- **Safe vault decryption**: raw crypto errors are never exposed. AES-256-GCM decrypt failures are caught, recorded as a `VAULT\_DECRYPT\_FAILURE` SIEM event, and returned as a generic 422.
- User-facing block/warning pages use **HTML escaping** + a strict Content-Security-Policy (**anti-XSS**); hop-by-hop headers are stripped before forwarding; **SSRF host blacklist**.

Known tradeoffs / future work

- Whitelist rules skip threat scanning: a user ALLOW rule bypasses VirusTotal by design; a whitelisted domain that later turns malicious stays allowed until the rule is removed.
- HTTPS is inspected at the hostname level only: for `CONNECT` tunnels the gateway checks domain reputation but cannot see the URL path without TLS interception. Scope limitation.
- Vault key lifetime: the derived key lives in process memory with no idle TTL until explicit lock/logout; in a multi-instance deployment each instance caches independently.
- Shared login/vault password: one credential protects both authentication and vault encryption (no separate vault passphrase).
- Plaintext metadata: vault service names and usernames are stored unencrypted (only the stored password is encrypted).
- Search and analytics at scale: traffic-log `urlSearch`/`destinationIp` use case-insensitive `contains` (ILIKE `%term%`) which can't use B-tree indexes — `pg\_trgm` would help if search becomes heavy. The analytics summary issues 10 grouped queries per call, bounded by a 90-day range cap.
- Per-request rule lookups: firewall rules are read from the DB on every proxied request (indexed). A cached rule set invalidated on CRUD would cut load under high traffic.

///// FILES EXPLANATION /////

➤ prisma/schema.prisma

User is the root entity. Sessions, FirewallRules, TrafficLogs, SecurityEvents and VaultCredentials belongs to a user. Deleting a user cleans up related data.

Models:

- User (id, username, passwordHash, kdfSalt, createdAt, lastAuthAt)
- Session (jwtId, expiresAt, revokedAt, lastVerifiedAt, ipAddress, userAgent, id, userId, createdAt)
Indexed: jwtId, userId
- FirewallRule (ruleType, pattern, action, enabled, id, userId, name, createdAt, updatedAt)
Indexed active rules for a user (@@index([userId, enabled])), userId
- TrafficLog (clientIp, url, destinationHost, policyDecision, threatVerdict, riskScore, machedRuleId, id, userId,

destinationPort, destinationIp, method, timestamp)

Indexed: timestamp, userId + timestamp, policyDecision, threatVerdict, riskScore, clientIp, destinationHost

- SecurityEvent (id, userId, type, severity, message, metadata, createdAt)

Indexed: type, severity, createdAt, userId + createdAt

- VaultCredential (encryptedPassword, IV, authTag, id, userId, serviceName, username, createdAt, updatedAt)

Indexed: @@index([userId, serviceName]), userId

➤ src/common

- guards - Can this request continue to the controller?

(request -> middleware -> guards -> interceptors -> controller)

- jwt-session.guard.ts:

export class JwtSessionGuard extends AuthGuard('jwt') - Passport JWT strategy (extracts JWT, verifies signature, validated expiration, loads user payload, attaches req.user)

- re-auth.guard.ts:

Did this user authenticate recently enough for sensitive operations?

async canActivate(context: ExecutionContext): Promise<boolean> {..} - depends on

JwtSessionGuard already running. Function fetches user from least DB state and checks

lastAuthAt, for computing elapsed time - too much time passed since password confirmation requires re-auth.

private logReAuthRequired(..) {..} - calls siemService.logSecurityEvent(..)

➤ utils

- jwt.util.ts:

This utility safely verifies JWT tokens and returns decoded payload if valid, null if invalid, instead of throwing exceptions.

This is the wrapper around jwtService.verifyAsync, with ignoreExpiration parameter to allow verifying even expired tokens. Used for token revocation cleanup, even when it is expired.

(expired but authentic token = valid payload, wrong JWT signature = rejected)

- password.util.ts:

Handles:

- Password hashing: *async function hashPassword(password: string): Promise<string> {..argon2.hash}*
- Password verification: *async function verifyPassword(password: string, passwordHash: string): Promise<boolean> {..argon2.verify}*
- Timing-safe authentication: Verifies a password while keeping the response time constant whether or not the account exists. When `passwordHash` is missing a full Argon2 verification against a dummy hash is run, preventing username enumeration via timing. It is computed once, first dummy verification matches the cost of a real one, good because Argon2 hashing is expensive and by this is avoided recomputing every request.

- request.context.util.ts:

Handles:

- Client IP extraction: *export function extractClientIp(req: Request): string*
- User-Agent extraction: *export function extractUserAgent(req: Request): string {..}*
- IP normalization: *function normalizeIp(value: string): string {..}* - returns IPv4 addresses format

➤ src/config

- auth.config.ts

Centralizes authentication and session security configuration, including JWT settings, secure cookie policies, and re-authentication timeout.

- env.util.ts

parsePositiveNumber(value: string | undefined, defaultValue: number) {}

- threat.config.ts

Handles threat-scanning integration settings, including VirusTotal API access, scan timeouts, and polling intervals for malware analysis.

➤ **policy.config.ts**

Defines centralized firewall and security policy rules that determine how the system reacts to malicious, suspicious, or safe traffic events.

➤ **proxy.config.ts**

Manages proxy server behavior, including network timeouts, temporary bypass tokens, and threat caching for performance optimization.

➤ **siem.config.ts**

Standardizes security event types and severity levels for SIEM-style monitoring, logging, and real-time security event tracking.

➤ **vault.config.ts**

Centralizes cryptographic vault settings: key derivation strength, encryption parameters, and secure key generation configuration.

➤ **throttle.config.ts**

Centralizes API throttling configuration. It protects the backend from abuse and brute-force attacks using configurable rate limits loaded from environment variables. Separated general API limits from authentication limits because login endpoints require stricter security.

➤ **cors.config.ts**

Centralizes CORS (Cross-Origin Resource Sharing) policy configuration. It defines which frontend applications are allowed to communicate with the backend and WebSocket gateway. Browsers block frontend apps from calling APIs on different domains unless explicitly allowed.

➤ **websocket.config.ts**

Centralizes WebSocket configuration and event definitions. The configuration handles namespace separation, CORS integration, and periodic system status broadcasting.

➤ **src/app.module.ts**

AppModule is the root NestJS module, main dependency container of the backend. It connects all infrastructure and backend modules: authentication, threat analysis, proxy services, SIEM monitoring, WebSocket communication, and global security middleware like rate limiting.

EventEmitterModule.forRoot() - allows services to communicate asynchronously

ThrottlerModule.forRoot({ - global rate limiting

ttl: throttleConfig.ttlMs(),

limit: throttleConfig.limit()

})

PrismaModule - ORM/database access layer

CommonModule - Shared reusable utilities/services (guards, types, utils, decorators)

SiemModule - Security monitoring/logging

ThreatModule - Threat intelligence / malware scanning

AuthModule - Authentication system, JWT/session management

HealthModule - System health, database status and VirusTotal API

RulesModule - Firewall/policy engine, allow/block decisions

VaultModule - Encrypted credential storage

ProxyRealmModule - Proxy auth prompt names and authentication metadata

ProxyModule - Main HTTP/HTTPS security proxy gateway

WebsocketModule - Realtime communication, live dashboard updates

➤ **src/main.ts**

The bootstrap file initializes the NestJS application, applies global security middleware, validation, and CORS policies, then starts both the management API and a custom authenticated proxy server used for traffic inspection.

import 'dotenv/config'; - loads .env automatically, variables become available globally
async function bootstrap(): Promise<void> {..} - entire backend app creation
bootstrap().catch(..) - error handling

```

async function bootstrap(): Promise<void> {
  const app = await NestFactory.create(AppModule); // Creates app from AppModule
  app.use(helmet()); // Adds secure HTTP headers:
  app.enableCors({
    origin: corsConfig.allowedOrigins(),
    credentials: true, // Enables browser communication with frontend,
                      // credentials: true - cookies/auth sessions allowed
  });
  app.use(cookieParser()); // Allows backend to read cookies from requests
  app.useGlobalPipes(
    new ValidationPipe({ // Automatically validates incoming DTOs:
      whitelist: true,    // - whitelist: true - Removes unknown properties
      forbidNonWhitelisted: true, // - forbidNonWhitelisted: true - Throws error immediately
      transform: true,    // - transform: true - Automatically converts request types ("123" -> 123)
    })
  );

  const apiPort = Number(process.env.PORT ?? 3000);
  await app.listen(apiPort);
  logger.log(`GuardMe API listening on port ${apiPort}`);

  const proxyServer = createProxyServer(app); // Creates HTTP proxy server
  const proxyPort = proxyConfig.port();
  await new Promise<void>((resolve, reject) => {
    proxyServer.once('error', reject);
    proxyServer.listen(proxyPort, resolve);
  });
  logger.log(`GuardMe proxy listening on port ${proxyPort}`);
}

```

function createProxyServer(app: INestApplication): Server {..} - creates a custom authenticated proxy server. Proxy receives browser traffic, authenticates the user, scans/filters requests, forwards allowed traffic and blocks malicious traffic.


```

function createProxyServer(app: INestApplication): Server {
  const proxyAuth = app.get(ProxyAuthService);
  const proxyPipeline = app.get(ProxyPipelineService);
  const connectTunnel = app.get(ConnectTunnelService);
  const proxyRealm = app.get(ProxyRealmService);

  const proxyApp = express();
  proxyApp.disable('x-powered-by');

  proxyApp.use((req: Request, res: Response) => {
    void (async () => {
      let user;
      try {
        user = await proxyAuth.authenticate(req);
      } catch {
        respondProxyAuthRequired(res, proxyRealm.getRealm());
        return;
      }

      await proxyPipeline.handleHttp(req, res, user);
    })();
  });

  const server = createServer(proxyApp);

  server.on('connect', (req: IncomingMessage, clientSocket: Socket) => {
    void (async () => {
      try {
        const user = await proxyAuth.authenticate(req);
        await connectTunnel.handle(req, clientSocket, user);
      } catch {
        const realm = proxyRealm.getRealm();
        clientSocket.write(
          'HTTP/1.1 407 Proxy Authentication Required\r\n' +
          'Proxy-Authenticate: Basic realm="${realm}"\r\n\r\n',
        );
        clientSocket.destroy();
      }
    })();
  });

  return server;
}

```

Uses NestJS dependency injection, services stays centralized, architecture modular and business logic is reusable.

ProxyAuthService - authenticates proxy users
ProxyPipelineService - processes HTTP requests
ConnectTunnelService - handles HTTPS tunneling
ProxyRealmService - provides proxy auth realm name

Removes X-Powered-By: Express header, preventing attackers from knowing technology stack

Catches all HTTP requests
Async wrapper - void avoids unhandled promise warnings

Before traffic is allowed proxy verifies credentials and validates session or user.
If authentication fails, `respondProxyAuthRequired` sends 407 Proxy Authentication Required.

When browser receives Proxy-Authenticate: Basic realm="GuardMe", it shows username and password popup.

Handles HTTPS tunneling (because Express HTTP server cant handle correctly).
HTTPS is encrypted and proxy cant directly read HTTPS requests.
Browsers use `CONNECT google.com:443 HTTP/1.1` and proxy opens raw TCP tunnel to the server (in this example Google HTTPS server).

By this app supports HTTPS browsing and encrypted traffic forwarding.

➤ src/modules

- auth - implements a layered authentication, combining JWT cookies, persistent session tracking, device fingerprint verification, SIEM security logging, real-time session events and re-authentication.

DTOs: AuthProfileDto, LogoutResponseDto, VerifyPasswordResponseDto; LoginDto, RegisterDto, VerifyPasswordDto - uses class-validator, DTO validation becomes automatic in ValidationPipe in main.ts

- strategies/jwt.strategy.ts:

Passport strategy for request authentication. From incoming requests, it extracts JWT, validates it, validate session and fingerprint, then if everything passes, attaches authenticated user.

- *async validate(req: Request, payload: JwtPayload): Promise<AuthenticatedUser> {}* - verifies session from DB too (*findActiveByJwtId(payload.jti)*), fingerprint verification (*verifySessionForRequest()*), checks user existence (*findById(payload.sub)*), updates session last verification (*updateLastVerifiedAt(session.id)*) and returns authenticated user (scoped auth context instead of DB entry).

- auth.controller.ts:

- POST /auth/register create account
- POST /auth/login authenticate
- POST /auth/logout revoke session
- GET /auth/me current profile

- POST /auth/verify-password re-authentication

Throttle authLimit for register, login and verify-password, protects against brute force.

JwtSessionGuard protects authenticated routes.

➤ auth.service.ts:

- *async register(dto: RegisterDto): Promise<PublicUserProfileDto> {}* - creates user and returns only public profile information while keeping authentication secrets isolated from API responses.

- *async login(dto: LoginDto, req: Request, res: Response): Promise<PublicUserProfileDto> {}* - Login uses timing-safe password verification against username enumeration. Creates server side session and stores it in database, enabling auditing and session management. Authentication tokens are stored in secure HTTP-only cookies. Cookie is configured with *httpOnly* (prevents JS access, reducing XSS), *secure* (HTTPS only) and *sameSite* (CSRF mitigation).

Authentication flow:

Username + Password --> Find User --> Timing-Safe Password Verification --> Create Session --> Issue JWT --> Store Cookie --> Rotate Proxy Realm --> Emit Login Event

- *async logout(req: Request, res: Response): Promise<LogoutResponseDto> {}* - Logout performs complete session cleanup, including token invalidation, vault key removal, cookie destruction, and security event emitting.

Flow:

Logout --> Read JWT --> Revoke Session --> Clear Cookie --> Clear Vault Key --> Rotate Proxy Realm --> Emit Logout Event

- *async getProfile(user: AuthenticatedUser): Promise<AuthProfileDto> {}* - Profile responses include session integrity (fingerprintMatched) and last authentication time.

- *async verifyPassword(user: AuthenticatedUser, password: string, req: Request):*

Promise<VerifyPasswordResponseDto> {} - Re-Authentication, used by sensitive operations.

Failed authentication and re-authentication attempts are forwarded to the SIEM, supporting security monitoring and incident investigation.

- *private async issueSessionCookie(userId, req, res) {}* - Session issuance combines JWT-based authentication with persistent session records.

- *private setSessionCookie(res, accessToken, expiresAt) {}*

- *private clearSessionCookie(res) {}*

- *private emitSessionEvent(type: SessionEventPayload['type'], userId: string, username: string) {}*

- *private resolveExpiryDate(expiresIn: string): Date {}*

//suggestions: refresh tokens for longer sessions (Access token + Refresh token)

➤ health - provides monitoring for internal and external dependencies, checking database connectivity and VirusTotal availability. Cached health responses to reduce external API calls.

DTO: SystemStatusPayload + HealthState

➤ health.controller.ts: GET /health

➤ system-status.service.ts:

- Private cash with expiration timestamp, to not burn VirusTotal quota (4req/min).

- *async getStatus(): Promise<SystemStatusPayload> {}* - checks DB and VT API health. When request arrives, is cache still valid? If yes, returns cached data. If not, calls in parallel fresh checks, updates cache and returns payload.

- *private async checkDatabase(): Promise<HealthState> {}* - DB connectivity test without expensive operations (lightweight *await this.prisma.\$queryRaw`SELECT 1`*).

- *private async checkVirusTotal(): Promise<HealthState> {}* - VT API external probe, with timeout for response.

➤ rules - manages user-defined firewall rules that control whether specific domains or IP addresses are allowed or blocked. Rule creation, updating, deletion, normalization, and runtime policy evaluation.

DTOs: CreateFirewallRuleDto, UpdateFirewallRuleDto, RulesListPayload

➤ utils/rule-matcher.util.ts:

- *normalizeRulePattern(ruleType, pattern)* - trims whitespace, removes control characters, limits length and normalizes domains.
CONTROL_CHARS = /[\x00-\x1F\x7F]/g; - prevents hidden characters.
.toLowerCase().replace(/\.\/, "/) - domain normalization; prevents *google.com*, *Google.com* and *google.com.* being treated differently.
- *matchesDomainRule(host, pattern)* - returns true/false if pattern matches the exact domain or subdomain (*return normalizedHost.endsWith(`.\${normalizedPattern}`)*) - allows *sub.google.com* to match *google.com*, but *evilgoogle.com* to not be matched).
- *matchesIpRule(hostOrIp, destinationIp, pattern)* - true/false if pattern matches resolved dest IP
- *ruleMatchesContext(rule, context)* - true/false if a firewall rule applies to the current network destination context

➤ rules.controller.ts:

For all API calls uses *JwtSessionGuard* which means only authenticated access to this, for sensitive operations (create/update/delete) *ReAuthGuard* as well. Sensitive firewall rule modifications require re-authentication to reduce the risk of unauthorized policy changes.

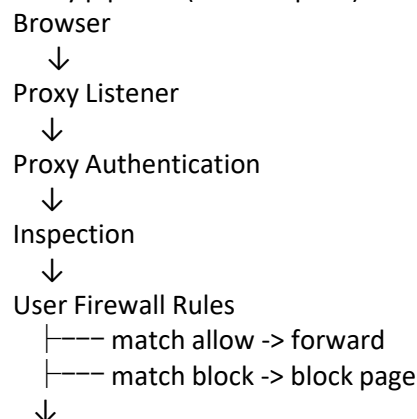
- GET /rules/list
- POST /rules/create
- PATCH /rules/update
- DELETE /rules/remove

➤ rules.service.ts:

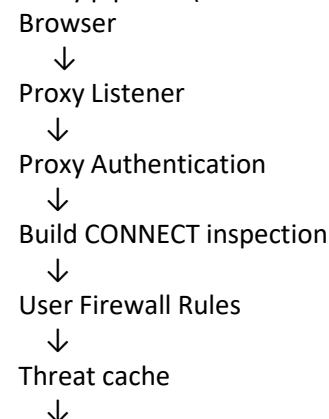
- *listForUser(userId)* - returns a list of user and system firewall policies.
- *createForUser(userId, dto)* - creates firewall rules after normalizing and validating rule patterns.
- *updateForUser(userId, ruleId, dto)* - updates use rule //suggested optimization: instead of multiple DB queries, collect it into single one fetch.
- *deleteForUser(userId, ruleId)* - deletes user defined rule policy.
- *evaluateRules(userId, context, resolveDestinationIp?)* - prioritizes block rules over allow rules to enforce secure default behavior.
- *withResolvedIp(rules, context, resolveDestinationIp?)* - Destination IP resolution is performed only when IP rules exist and destination IP is not already known, reducing DNS overhead.
- *assertRuleOwnedByUser(userId, ruleId)* - access control, users can only modify their own rules.
- *resolvePattern(ruleType, pattern)* - wraps internal normalization failures into API-friendly errors.
//suggestions: CIDR support for whole subnets, not just exact IP only, conflict detection (for better UX, maybe rule ordering, now BLOCK > ALLOW), scalability: rule evaluation in memory after loading all rules from DB, for larger scale indexed matching, different rule engine...

- **proxy** - security gateway that for every HTTP and HTTPs request authenticates users, inspects requests, evaluates user firewall rules, performs threat intelligence scanning, applies security policies, logs everything to a SIEM, and only then forwards traffic to the destination server.

Proxy pipeline (HTTP request):



Proxy pipeline (CONNECT - HTTPS):



Threat cache



Threat scan



Policy Engine

----- LOG + BLOCK -> block page

----- LOG + WARN -> warning page

----- LOG + ALLOW -> proxy forward

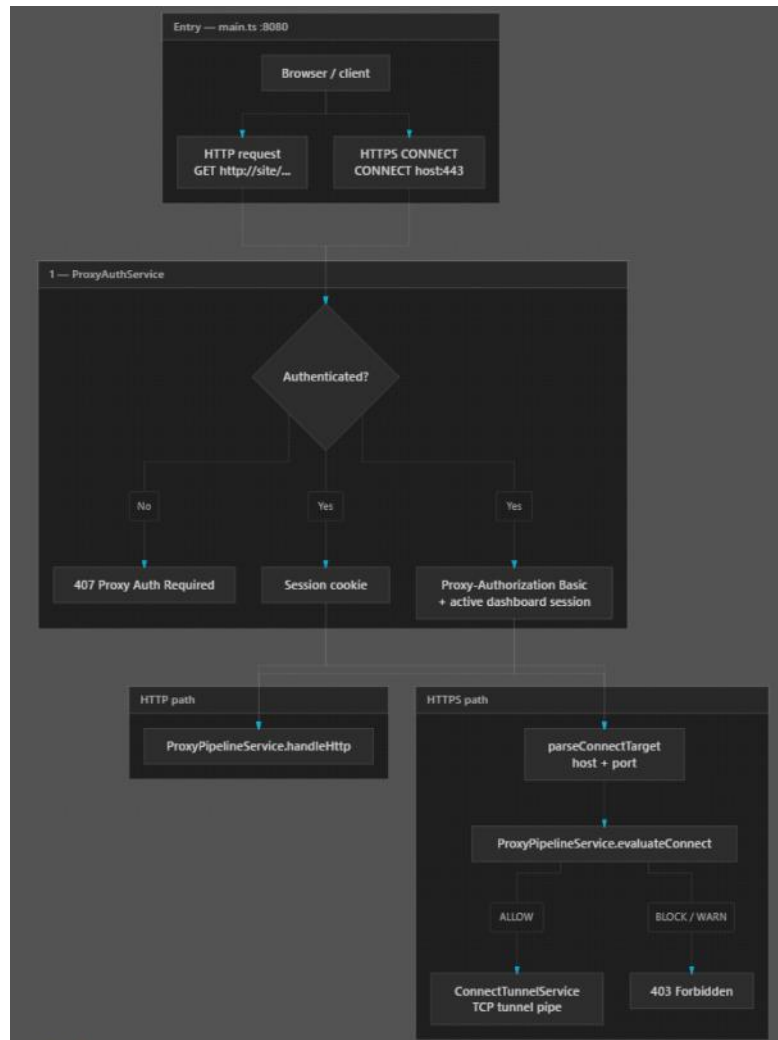
Threat scan

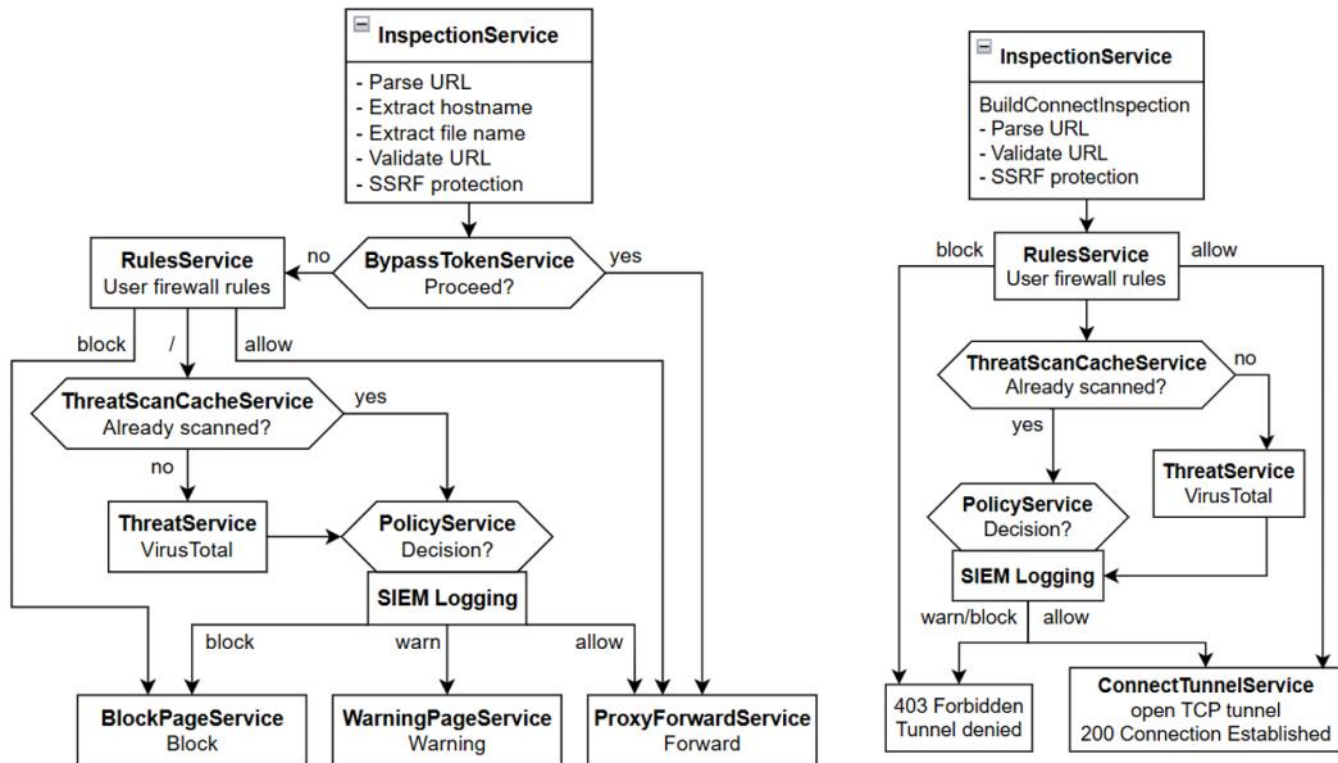


Decision (Policy engine)



Open TCP tunnel





DTOs: InspectionResult (normalized object instead of passing Express Request everywhere); PolicyResult (abstracted VirusTotal details); BlockPageContext, WarningPageContext (presentation doesn't know about threat scan result and it only receives what it needs).

Templates: block.html, warn.html - protected against XSS by:

- CSP (Content Security Policy) that tells the browser to not execute JS (CSP forbids scripts: *Content-Security-Policy: default-src 'none'; style-src 'unsafe-inline';*)
- HTML escaping to prevent HTML injection by converting HTML characters into safe entities.

Malicious URL: `https[:]//evil.com/<script>alert(1)</script>` displaying URL on page:

```
<div>
```

```
    https[:]//evil.com/<script>alert(1)</script>
```

```
</div>
```

But with escaping it becomes: and browser reads it as a plain text

```
<div>
```

```
    https[:]//evil.com/&lt;script&gt;
```

```
</div>
```

➤ utils

➤ connect-target.util.ts:

Parses and validates the target destination from HTTPS CONNECT requests, extracting the host and port. It rejects malformed or invalid destinations before the proxy attempts to establish a connection.

➤ dns.util.ts:

Converts a hostname into its IP address using DNS with a 2-second timeout.

➤ hop-by-hop-headers.util.ts:

Removes hop-by-hop HTTP headers before forwarding requests and responses. These are headers that should never be forwarded by proxies to not make connection-related issues between clients and servers.

➤ html-escape.util.ts:

Escapes user-controlled values - dangerous HTML characters, before inserting them into HTML templates, preventing Cross-Site Scripting (XSS) attacks on proxy-generated pages.

➤ proceed-url.util.ts:

Generates bypass URLs by attaching a secure verification token to the original request. This allows users to proceed warning pages.

`https[:]//warning.com --> https[:]//warning.com?guardme_bypass=TOKEN`. The token tells the proxy the user already accepted the warning.

- proxy-basic-auth.util.ts:
Decodes and validates HTTP Basic proxy authentication credentials from incoming requests, allowing the proxy to authenticate connecting clients.
- template-render.util.ts:
Loads HTML templates and safely injects dynamic values into them. All inserted content is HTML-escaped to prevent malicious script injection.
- threat-summary.util.ts:
Converts detailed threat scan results into concise readable summaries (n malicious, n suspicious, n harmless).
- url.util.ts:
Removes and reads the bypass token from the URL. Checks if the proxy is trying to access a private destination addresses, to stop accessing internal systems and to protect against Server-Side Request Forgery (SSRF) attacks.
- proxy-auth.service.ts: - authenticates the user
Authenticates proxy users through dashboard sessions or proxy credentials while enforcing rate limiting and session validation.
- inspection.service.ts: - validates and normalizes the request, including SSRF checks
Parses and validates incoming requests, extracts destination information, blocks unsafe targets, and detects potential file downloads.
- bypass-token.service.ts: - checks whether a warning was previously acknowledged
Issues and validates one-time JWT tokens that allow users to bypass warning pages safely.
- threat-scan-cache.service.ts: - reuses recent scan results when available
Temporarily caches threat scan results to reduce repeated calls to the threat intelligence provider.
- policy.service.ts: - converts scan results into an allow, warn, or block decision
Converts threat intelligence and file scan results into allow, warn, or block decisions.
- proxy-pipeline.service.ts: - orchestrates the overall flow and coordinates the services above
Coordinates the entire security pipeline by orchestrating inspection, scanning, policy evaluation, logging, and forwarding.
- proxy-forward.service.ts: - securely forwards approved requests to the destination
Securely forwards approved requests and streams responses back to the client while sanitizing HTTP headers.
- block-page.service.ts: - generate the appropriate HTML response for blocked requests
Generates a secure HTML page explaining why access was denied.
- warning-page.service.ts: - generate the appropriate HTML response for warned requests
Generates an interstitial warning page that allows the user to proceed after acknowledging the security risk.
- connect-tunnel.service.ts: - handles HTTPS CONNECT requests by evaluating the destination before creating a TCP tunnel
Handles HTTPS CONNECT requests by evaluating the destination before establishing a secure TCP tunnel.
- proxy-rate-limit.service.ts: - provide supporting authentication features: rate limiting
Protects the proxy against brute-force authentication attempts by tracking failed logins per client IP.
- proxy-realm.service.ts - provide supporting authentication feature: proxy authentication realm
Maintains the authentication realm used to prompt browsers for proxy credentials.

Service	Role
ProxyAuthService	Cookie or Basic auth; 407 if missing
ProxyPipelineService	Orchestrates the whole pipeline
InspectionService	URL parse, SSRF blocklist, file heuristic (HTTP only)
BypassTokenService	Sign / verify one-time "Proceed" tokens (HTTP only)
RulesService	Per-user ALLOW/BLOCK rules (before threat scan)
ThreatScanCacheService	In-memory VT result cache per URL
ThreatService	VirusTotal integration
PolicyService	Verdict → ALLOW / WARN / BLOCK
SiemService	traffic_logs + security_events + WebSocket emit
BlockPageService	HTML block page
WarningPageService	HTML warning + proceed link (HTTP only)
ProxyForwardService	Forward HTTP to upstream (axios)
ConnectTunnelService	Raw TCP tunnel for HTTPS

HTTP vs HTTPS — important differences

	HTTP	HTTPS (CONNECT)
What proxy sees	Full URL, method, headers	Hostname + port only
Threat scan	Full URL	Synthetic https://host:port/
File scan	Yes (extension heuristic)	No
Warning page	Yes (WARN + bypass token)	No — WARN → 403
Bypass token	Yes	No
User ALLOW rule	Forward, skip VT	Open tunnel, skip VT
On ALLOW	ProxyForwardService	ConnectTunnelService TCP pipe

- **sessions** - manages session lifecycle with persistent JWT tracking, device fingerprint verification, automatic session revocation, and SIEM-integrated security event monitoring
 - DTOs: CreateSessionInput, SessionVerificationResult
 - **session.service.ts:**
 - *create(input: CreateSessionInput): Promise<Session> {}* - stores session metadata
 - *findActiveByJwtId(jwtId: string): Promise<Session | null> {}* - returns only a valid session
 - *findLatestActiveForUser(userId: string): Promise<Session | null> {}* - useful for re-auth logic
 - *revokeByJwtId(jwtId)* and *revokeAllForUser(userId)* - session expiration control
 - *async updateLastVerifiedAt(sessionId: string): Promise<void> {}* - tracks session activity
 - *async verifySessionForRequest(session: Session, req: Request): Promise<SessionVerificationResult> {..}* - validates if fingerprint matches with IP and UserAgent stored in database for that session. If not, SIEM logs security event (mismatch). If sessionFingerprintStrict is configured on true, session is revoked and SIEM logs this as well.
- **siem** - provides centralized security logging with real-time event streaming integration. User-scoped analytics with pagination, filtering and time-series.
 - DTOs: TrafficLogsQueryDto, SecurityEventsQueryDto, AnalyticsSummaryQueryDto, Analytics Payload
 - DTOs (defines traffic statistics, risk statistics, top destinations, event summaries)

- utils/analytics-agregation.util.ts:
 - *function toCountRecord()* - Converts database group-by results into predictable structures, frontend never needs null checks.
 - *function roundRiskAverage()* - Rounds a number. (71.3333 -> 71.3)
- utils/analytics-range.util.ts:
 - *function resolveAnalyticsRange()* - Safely handles analytics date ranges, validates chronology and automatically selects bucket size, default 7 days, max 90 days.
- utils/search-term.util.ts:
 - *function sanitizeSearchTerm()* - Search inputs are sanitized before database filtering, by trimming, limits the length of string and removing control characters.
- siem-analytics.service.ts:
 - Traffic Analytics - transforms raw traffic data into actionable metrics such as request volumes, risk distributions, policy outcomes, and frequently accessed destination. Calculates total requests, policy decisions distribution (how many ALLOW, how many BLOCK), threat verdicts, risk statistics (average, max, high risk requests count) and top hosts (most accessed destinations).
 - Security events Analytics - aggregated into severity and category breakdowns (event types) to provide visibility into total count and threat activity.
 - Time-series aggregation enables visualization of security and traffic across hourly, 6-hour, and daily intervals.
 - Analytics calculations execute in parallel to minimize dashboard response time (*Promise.all(...)*).
- siem.controller.ts:

All SIEM API calls are authenticated and scoped per user. All protected by JwtSessionGuard.

 - GET /siem/traffic-logs
 - GET /siem/security-events
 - GET /siem/analytics/summary
- siem.module.ts:

The SIEM module is registered globally *@Global()* because many modules needs SIEM logging. (Auth, Vault, Proxy, Threat Detection, Sessions)
- siem.service.ts:

The SIEM service uses an event-driven architecture. (*eventEmitter.emit(...)*)

 - *async logTraffic(input: TrafficLogInput): Promise<void> {}* - Saves traffic log in DB and emits. Every proxied request is recorded with policy decisions, threat verdicts, and risk scores, creating a complete audit trail for investigations.
 - *async logSecurityEvent(input: SecurityEventInput): Promise<void> {}* - Saves security event in DB and emits. Severity is assigned automatically (*resolveEventSeverity(..)*).
 - *async findTrafficLogsForUser(userId, query)* - Query API for traffic logs, includes pagination, filtering, sorting. Uses *private buildTrafficLogWhere(userId, query)*.
 - *async findSecurityEventsForUser(userId, query)* - Query API for security events, includes pagination, filtering, sorting. Uses *private buildSecurityEvent(userId, query)*.
 - *private logPersistenceFallback() {}* - If DB fails, *logger.warn(..)* instead of crashing proxy flow. Logging failures are handled by logging, to ensure monitoring issues do not interrupt core proxy functionality.
- **threat** - responsible for scanning URLs using the VirusTotal threat intelligence API. Checks if the URL is safe, suspicious, malicious or unverified. A provider layer for external API communication, and a service layer that handles business logic, fail-safe behavior, and SIEM logging.

DTOs: ThreatScanActor, ThreatScanResult, ThreatVerdict
- providers/virustotal.provider.ts:
 - *async scanUrl(url: string): Promise<ThreatScanResult> {}* - main function, first checking for cached VirusTotal results and then performing live analysis if necessary. (try cached report, return if exists, if not, submit URL and poll analysis until complete, map result into internal format and return verdict).

- *tryGetCachedReport(url, deadline)* - tries to get existing VT report to reduce unnecessary API calls.
- *submitUrlForAnalysis(url, deadline)* - submits a URL for fresh scanning and receives an analysis identifier.
- *pollAnalysisResult(url, analysisId, deadline)* - continuously checks analysis until scanning finishes or a timeout occurs.
- *mapStatsToResult(scannedUrl, stats, metadata)* - converts VT analysis statistics into internal verdicts.
 (malicious > 0 => MALICIOUS
 suspicious > 0 => SUSPICIOUS
 otherwise => SAFE) #suggestion: for more accurate risk scoring - malicious / total engines
- threat.module.ts:
 - HTTP communication settings:
 timeout: *threatConfig.scanTimeoutMs()* - timeout for waiting response after sent HTTP request
 maxRedirects: 0 - prevent automatic redirect following, ensuring the gateway only communicates with explicitly intended URL (VT endpoint).
- threat.service.ts:
 - *async scanUrl(url: string, actor?: ThreatScanActor): Promise<ThreatScanResult> {}* - main function manages the threat scanning process. Fail safe strategy for system to remain operational even if the external provider fails. (If VT API is unavailable or times out, allow traffic as UNVERIFIED while recording security events in the SIEM system).
- users - manages account creation and fetching, applying password hashing and cryptographic salt generation, DTO-based protection to prevent sensitive user data exposure
 DTOs: *CreateUserInput*, *PublicUserProfileDto*
- user.service.ts:
 - *findByUsername()* and *findById()* - get user
 - *async create(input: CreateUserInput): Promise<User> {}* - user registration
 - *async updateLastAuthAt(userId: string): Promise<void> {}* - tracks last successful authentication
 - *toPublicProfile(user: User): PublicUserProfileDto {}* - transforms DB entity in safe API response
- vault - credentials are encrypted with a key derived from the user's password and the decrypted key is only temporarily in server memory after an explicit unlock.
 DTOs: *CreateVaultCredentialDto*, *UpdateVaultCredentialDto*, *UnlockVaultDto*, *Vault Payload*
 Interfaces - define API response payloads, exposing only the data required for each operation, supporting least-privilege (password only returned when explicitly requested).
- guards/vault-unlock.guard.ts:

Protects encrypted data access. It checks if the user is authenticated and is vault unlocked (vault key exists in memory cache), providing extra security boundary beyond authentication by requiring the vault to be explicitly unlocked before protected credential operations.
- crypto.service.ts:

Vault encryption keys are derived from user passwords using Argon2id. Instead of using user password directly, 256-bit vault key is generated based on user password + salt (unique to each user).

Credentials are encrypted using AES-256-GCM with random initialization vector (iv) for each credential (this makes encrypted passwords unique even if original passwords are the same - same user (user password), different services with same service passwords). In database are stored: encrypted password, iv and auth tag.
- vault-key-cache.service.ts:

In-memory vault unlock mechanism. It stores *userId: derived vault key*. Unlock stores key (*setKey()*), lock sets 0 and deletes (*clearKey(... key.fill(0)...)*).
- vault.controller.ts:

Applies layered security controls including session authentication, vault unlock verification, re-authentication checks, and request throttling for sensitive operations. Every API call is protected by JwtSessionGuard, sensitive operations + VaultUnlockGuard, highly sensitive operations + ReAuthGuard.

- GET /vault/status
- POST /vault/unlock
- POST /vault/lock
- POST /vault/credentials
- GET /vault/credentials
- GET /vault/credentials/:id
- PATCH /vault/credentials/:id
- DELETE /vault/credentials/:id

➤ vault.service.ts:

- *unlock(user, password, req)* - Vault unlocking requires successful password verification, derives a cryptographic vault key using Argon2id, caches it in memory for the active session, and generates SIEM events for both successful and failed attempts.

Flow:

Password submitted --> Verify password hash --> Generate vault key --> Store key in memory --> Update re-auth timestamp --> Log SIEM event

- *createCredential(userId, dto)* - Credential creation encrypts sensitive secrets before persistence, ensuring plaintext passwords are never stored in the database.
- *getCredential(userId, credentialId)* - Credential retrieval performs decryption using the user's vault key and detects AES-GCM integrity validation, with failures logged to the SIEM.

Flow:

Load encrypted data --> Load vault key --> Decrypt --> Return password

- *findOwnedCredential(userId, credentialId)* - Every vault operation (get/update/delete) enforces ownership validation at the database layer, ensuring users can only access credentials associated with their own account.
- *lock(userId)*
- *listCredentials(userId)*
- *updateCredential(userId, credentialId, dto)*
- *deleteCredential(userId, credentialId)*

➤ websocket - provides real-time delivery of security events, traffic logs, session activity, and system health updates, enabling live monitoring without requiring client polling

DTOs define what frontend receives: TrafficLogPayload, SecurityEventPayload, SessionEventPayload

➤ utils/payload-mapper.util.ts:

Mapper utilities separate database models from external API, controlling data exposure

- *toTrafficLogPayload()*
- *toSecurityEventPayload()*

➤ ws-auth.service.ts:

Authenticates every WebSocket connection. Every connection validates the server-side session, preventing access through revoked or expired credentials. It also verifies device fingerprint. (adding protection against session theft and unauthorized token reuse)

Flow:

WebSocket Connect --> Extract JWT Cookie --> Verify JWT --> Verify Session --> Verify Fingerprint --> Load User --> Accept Connection

- *async authenticateSocket(client: Socket): Promise<AuthenticatedUser> {}* - uses:
- *private extractCookieToken(cookieHeader: string | undefined): string | null {}* - WebSocket connections reuse the application's secure session infrastructure, ensuring consistent authentication and authorization across both HTTP and real-time channels.

➤ events.gateway.ts:

The gateway manages the full lifecycle of authenticated real-time connections, including

initialization, connection handling, periodic broadcasts and cleanup.

- *async handleConnection(client: AuthenticatedSocket) {}* - on connect *authenticateSocket()* and joins the room, on failure disconnect client. Clients are authenticated before receiving any events, and unauthorized connections are immediately terminated to prevent access. Socket.IO rooms provide user-level event isolation, ensuring that sensitive security and traffic information is delivered only to the intended recipient.

Each user gets their own event stream, user room *user:\${userId}* (example user:123, user:99) and joins shared authenticated room for system status.

- The gateway follows an event-driven architecture, consuming internal application events and transforming them into real-time client updates - SIEM service doesn't know about WebSocket.
 - *@OnEvent(SIEM_EMIT_EVENTS.TRAFFIC_LOG)*
 - *@OnEvent(SIEM_EMIT_EVENTS.SECURITY_EVENT)*
 - *@OnEvent(SESSION_EVENT)*